Solar Elevation Estimation from Single Images Using Convolutional Neural Networks for Multi-Image Geolocation

[Alex Nowlan]
Department of Computer Science and Software Engineering
University of Canterbury, Christchurch, New Zealand
[ano58@uclive.ac.nz]

***Abstract*—We present a 2 stage pipeline for estimating the goegraphic location of a set of imaes using only the images content and the timestamp at time of capture. The first stage uses an EfficientNet-B0 convolutional neural network, fine tuned on 27,848 outdoor images sourced from Google stree view panoramas, predicts the elevation of the sun in an image. The model is trained using a soft classification objective over 32 discrete bins, acheiving a mean absolute error of 0.511° and an RMS error of 0.753° which places 98.3% of predictions within $\pm 2^\circ$ of ground truth on an evaluation image set. The second stage uses the predicted elevation plus an accurate timestamp to compute a circle of candidate observer locations on the globe. Intersecting the circles from $K \geq 2$ images seperated in time yeilds two points on the globe where the image may have origionated from, reflected over the equator. Despite strong CNN performance, the second stage fails to provide accurate geolocation of images with error in the range of thousands of killometers. We discuss sensitivity of stage two to noise in image timestamp and identify this as a potential bettleneck for practical performance.**

***Index Terms*—solar elevation, geolocation, convolutional neural network, sun position model, EfficientNet-B0**

I. Introduction

The automatic determination of where a photograph was taken, without relying on embedded GPS metadata is an open problem in computer vision, with applications in digital forensics [1], investigative journalism, and autonomous navigation.

Most online photographs are not tagged with geolocation metadata, and most photographers do not have the time, patience, or will to manually label the images they take. Even so, many images taken on mobile devices include a geotag [2], but this data is often stripped when an image is uploaded to the internet.

Current methods of geolocating images focus on landmark recognition requiring a global landmark index, or manual labor in labeling image data. These methods may work well but require a large amount of labor or data. In this paper, we propose a simpler, automated design.

Most modern photography includes very accurate timestamps for when an image was taken [3] and is often included in image metadata without a photographers knowledge. Using this timestamp, it is possible to extrapolate the solar position of the sun as the sun's apparent position in the sky is a deterministic function on time.

Given this solar position, the elevation of the sun in an image can be measured to determine the observers distance from the subsolar pont (the point on earth where the sun is directly overhead for a given time). Since the subsolar point and distance from this point is know, we can infer that an observer must exist on a circle on the globe centered at the subsolar point with a radius proportional to the observed solar elevation.

All that is required to narrow the search space is to use multiple images to determine multiple potential circle sets, then take the intersection of the those circles to find two unique points on earth where an observer must be located.

We propose that a Convolutional Neural Network (CNN) is able to learn the features of an image and predict the elevation of the sun in the sky. We fine tune the existing CNN, EfficientNet-B0 originally trained on the images segmentation dataset ImageNet-1K, to learn solar signatures. We then create a geolocation pipeline that takes only a set of images (#images $\geq$ 2) and their corresponding timestamps to approximate geographic location of said image set.

The remainder of this paper is structured as follows:

Related work will cover previous attempts at solving this problem, comparing and contrasting the differences to our approach.

The methodology section explains the full pipeline in detail as well as all of its subsystems and design choices.

We then analyze the results and explore notable findings from the CNN and Geolocation pipeline.

II. Related Work

A. Image-Based Geolocation

Attempts at geolocation have been made using only image data [4]. The Method *PlaNet* [5] attempts to use a CNN to guess the location of an image by splitting the globe into a large number of grid cells and training the model to estimate the closest cell.

This method has shown promise but requires the CNN to learn the entire landscape of earth for general accuracy. Our method differs in the fact that we assume more information is available to the model, namely, timestamps, and we use this to create a general purpose geolocator.

Newer methods using multi-modal Large language models (LLM's) [6] [7] have shown promising results at image geolocation, suggesting that LLM's are able to gain expert-level ability on tasks outside of their target domain. this paper showed GTP-4o [8] was able to predict the location of an images source with a high accuracy.

As above, LLM's also require huge sources of training data and have effectively scraped the entire internet to yield this result.

B. Solar Cues in Computer Vision

The paper by Frode Eika Sandnes [9], while not computer vision based, provides the backbone of our work in this paper. They show that using only solar elevation and an accurate timestamp, location of an image can be approximated. The use the fact that the suns elevation can be determined by looking only at the ratio between an objects height, and the length of its shadow.

We take this paper one step further by predicting the suns elevation with a CNN rather than manually measurement of images.

C. Camera Interference

Jisoo Choi et al [10] show that electrostatic interference caused by mains power lines effects camera sensors to te point that the specific frequencies of such interference can be measured from an image alone. the use this finding to label images to the two most common mains power frequencies, 50Hz and 60Hz. This does not fully geolocate an image, but greatly narrows the search area.

Based on the previous work, no serious attempt has been made at a general, fully autonomous image geolocation method. The paper by Frode Eika Sandnes is used as the backbone of out method with the addition of an autonomous, CNN based sun elevation estimation.

III. Methodology

A. System Overview

The proposed pipeline combines a learned image model with a deterministic geometric model of the Sun–Earth relationship. Given an outdoor photograph with a known timestamp, the system proceeds in two stages. First, a CNN estimates the solar elevation angle $\hat{\theta}$ visible in the scene. Second, the timestamp is used to compute the *subsolar point* (SP), the location on Earth's surface directly beneath the Sun at that instant. Together, the estimated elevation and the SP define a circle of candidate observer locations on the globe: every point on this circle would observe the Sun at the predicted elevation angle at that moment in time.

When multiple photographs are available from the same approximate location but captured at different times of year, each image yields a distinct candidate circle. The true observer location must satisfy all circles simultaneously, so the intersection of these loci is computed to recover a pair of candidate locations, which are reflections of one another across the equator. This ambiguity can often be resolved using contextual cues such as timezone information embedded in the image timestamp, or by discarding candidates that fall over open ocean when the scene clearly depicts land.

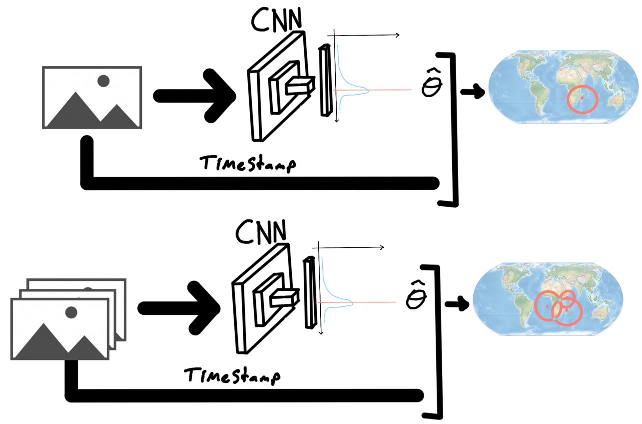

Fig. 1: System pipeline. A CNN estimates solar elevation $\hat{\theta}$ from each input image. Given timestamps, the sun-position model inverts the elevation to candidate (latitude, longitude) sets. Multiple images constrain the solution to a unique geographic location.

B. CNN Architecture and Transfer Learning

EfficientNet-B0 [11], pretrained on ImageNet-1K [12], is used as the backbone for solar elevation estimation. Transfer learning from ImageNet is particularly well-suited to this task [13] [14] as the pretrained weights encode general purpose visual features such as edge detection, texture recognition, and shadow geometry that are directly relevant to interpreting sun position from an outdoor scene. Fine-tuning from this initialization requires far less labelled data than training from scratch, and converges more stably.

Rather than framing solar elevation estimation as direct regression to a scalar angle, we treat it as a soft classification problem over 32 uniformly spaced bins spanning 0° to 90°. The original ImageNet classification head is replaced with a lightweight regression head:

$$\texttt{Dropout}(0.2) \rightarrow \texttt{Linear}(d_{\text{feat}}, 32),$$

where d_{feat} is the dimensionality of EfficientNet-B0's penultimate feature vector. The backbone network f_ϕ therefore maps an input image $\mathbf{I} \in \mathbb{R}^{H\times W\times 3}$ to a vector of 32 logits, one per bin:

$$\mathbf{z} = f_\phi(\mathbf{I}), \quad \mathbf{z} \in \mathbb{R}^{32}. \tag{1}$$

Each bin covers $\frac{90}{32} \approx 2.81°$ of elevation. Rather than committing to a single bin at inference time, the predicted elevation is computed as the expected value of the softmax distribution over bin centres $c_i = \frac{90(i-1)}{31}$:

$$\hat{\theta} = \sum_{i=1}^{32} \text{softmax}(\mathbf{z})_i \cdot c_i. \tag{2}$$

This soft expectation yields a continuous, smooth elevation estimate and avoids the quantization artefacts that would arise from a hard argmax over bins.

During training, each ground-truth elevation θ is encoded as a Gaussian soft label centred on the nearest bin (Section III-C), and the model is optimized by minimizing the Kullback–Leibler divergence (KL divergence) between the predicted log-probability distribution and the soft target:

$$\mathcal{L}_{\mathrm{KL}} = \sum_{i=1}^{32} P_i \log \frac{P_i}{Q_i}, \tag{3}$$

where P_i is the soft-label target probability for bin i and $Q_i = \mathrm{softmax}(\mathbf{z})_i$ is the model's predicted probability. Using KL divergence with soft labels is preferable to mean-squared error on raw angles because it allows the model to express calibrated uncertainty across adjacent bins, and penalises confident wrong predictions more heavily than uncertain ones. A full summary of training hyperparameters is given in Table I.

C. Dataset and Ground-Truth Labels

Training data was sourced from the Google Street View API[1], which provides equirectangular panorama images alongside accurate GPS coordinates and capture timestamps. A total of 6,467 panoramas were collected and manually reviewed to assess whether the Sun was visible in the scene. Panoramas in which the Sun was obscured by cloud cover, or in which the solar disc was not clearly visible, were discarded. This filtering step yielded 2,404 panoramas with reliable ground-truth solar elevation angles, which were manually labelled by use of a simple python script and a few hours of work.

To maximise the diversity of training perspectives from each panorama, In order to mimic real image conditions, 12 perspective were rendered per panorama using randomized camera parameters:

- Field of view: FOV $\in [50°, 90°]$,
- Yaw: $\in [0°, 360°]$ (uniform random),
- Pitch: $\in [-30°, 30°]$ relative to the horizon.

This strategy substantially increases the effective dataset size and exposes the model to a wide range of sky fractions, horizon positions, and shadow angles for each underlying scene. The final dataset comprised 28,848 perspective images, each inheriting the ground-truth solar elevation of its parent panorama.

An evaluation set of 1,000 images was reserved prior to training and was not used in training at any point, yielding a training set of 27,848 images.

Ground-truth elevation angles are not used as scalar regression targets directly. Instead, each angle θ is converted to a Gaussian soft label over the 32 elevation bins with standard deviation $\sigma = 2.5°$:

$$P_i = \frac{1}{Z} \exp\left(-\frac{(c_i - \theta)^2}{2\sigma^2} \right), \tag{4}$$

[1]https://developers.google.com/maps/documentation/streetview/overview

where c_i is the centre of bin i and Z is a normalisation constant ensuring $\sum_i P_i = 1$. With $\sigma = 2.5°$, meaningful probability mass is assigned to all bins within roughly $\pm 5°$ of the true elevation, this smooths the training signal across adjacent bins and preventing the model from being penalised for predictions that are close but not exact.

D. Sun–Earth Geometric Model

The geolocation stage relies on a deterministic model of the Sun's apparent position as a function of time and observer location. The key quantity is the *subsolar point* (SP), the geographic coordinate (δ, λ_s) at which the Sun is directly overhead for a given UTC timestamp.

The latitude of the SP, declination δ, varies seasonally and can be approximated as:

$$\delta = -0.4092797 \cos\left(\frac{2\pi}{365}(M + 10) \right), \tag{5}$$

where M is the day of the year and the constant 0.4092797 is the radian value for maximum solar declination, $23.45°$.

The longitude of the subsolar point λ_s follows directly from the UTC time τ (in hours):

$$\lambda_s = \frac{360\,\tau}{24}. \tag{6}$$

Given the subsolar point (δ, λ_s) and a measured solar elevation e at observer location (ϕ, λ), the elevation can be computed via the spherical law of cosines:

$$\sin(e) = \sin(\phi)\sin(\delta) + \cos(\phi)\cos(\delta)\cos(\lambda - \lambda_s). \tag{7}$$

Notice, for fixed e and (δ, λ_s), all solutions (ϕ, λ) to Equation 7 form a small circle on the surface of the sphere centred on the subsolar point. This circle represents the complete set of observer locations from which the Sun would appear at elevation e at the given moment in time.

E. Geolocation via Multi-Image Constraint Solving

A single elevation measurement constrains the observer to a circle on the globe, but cannot pinpoint a unique location. However, if $K \geq 2$ photographs are available from the same location, captured at different times of year, each image yields a distinct candidate circle set with a different centre (subsolar point) and radius (determined by the predicted elevation). Because all images share the same true observer location, that location must lie on every circle simultaneously. The intersection of two circles on a sphere generally produces exactly two points, which are reflections of one another across the equator.

The optimal observer location $(\hat{\phi}, \hat{\lambda})$ is recovered by minimising the sum of squared residuals between the elevation predicted by the geometric model and the elevation estimated by the CNN for each image:

$$(\hat{\phi}, \hat{\lambda}) = \arg\min_{\phi,\lambda} \sum_{k=1}^{K} \left(g(\phi, \lambda, t_k) - \hat{\theta}_k \right)^2, \tag{8}$$

where $g(\phi, \lambda, t_k)$ denotes the solar elevation predicted by Equation 7 for observer (ϕ, λ) at time t_k, and $\hat{\theta}_k$ is the CNN's elevation estimate for image k. This least-squares problem is solved over a dense grid of candidate locations and refined using gradient-free optimisation.

The two symmetric solutions arising from the equator ambiguity can often be resolved without additional images. If the timestamp carries an explicit timezone (e.g. NZST), only candidate locations within that timezone need be to considered, frequently eliminating one of the two solutions. Alternatively, if one candidate falls in open ocean and the scene clearly contradicts this, it can be discarded on geographic grounds.

F. Implementation Details

TABLE I: Implementation hyperparameters

Parameter	Value
Backbone	EfficientNet (ImageNet-1K pretrained)
Input resolution	224×224
Batch size	16
Optimiser	Adam ($lr = 1e-4$)
Epochs	30
Hardware	NVIDIA GeForce RTX 4060 Ti

IV. Experiments and Results

A. Evaluation Metrics

Three evaluation metrics are used, corresponding to the two stages of the pipeline. For the CNN elevation estimator, we report Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), both in degrees, measured on the 1,000-image evaluation set. MAE gives an average error in physical units, while RMSE penalises large outliers and draws attention to the tail of the error distribution. We also report the percentage of predictions falling within $\pm 2°$ and $\pm 5°$ of the true elevation, which demonstrates practical accuracy.

For the geolocation pipeline, performance is reported as the distance in kilometers between the predicted observer location and the true GPS coordinate from which the image set was captured.

B. CNN Solar Elevation Estimation

The CNN achieves a MAE of $0.511°$ and an RMSE of $0.753°$ on the evaluation set, representing strong generalisation given the $2.81°$ bin width of the output layer. The scatter plot of predicted versus ground-truth elevation (Fig. 3) shows predictions closely tracking the ideal diagonal across the full $0°$–$90°$ range, with no bias at either low or high elevation angles. This indicates that the model has successfully generalised across the full range of solar positions rather than guessing an average prediction.

The residual histogram (Fig. 4) confirms that the error distribution is sharply concentrated around zero, with a mean bias of only $0.069°$, indicating that over and under predictions are nearly symmetric. preciesly, 98.3% of predictions fall within $\pm 2°$ and 99.8% within $\pm 5°$ of the true elevation (Table II), confirming that large errors are rare.

We also analyse the CNN error as a function of true elevation with sample count per elevation bin overlayed in

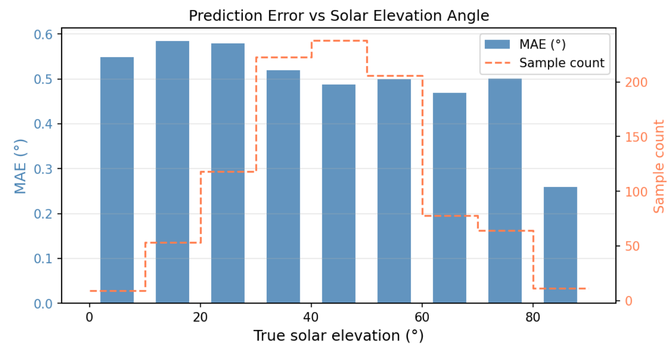

Fig. 2: Ground truth elevation vs. predicted elevation

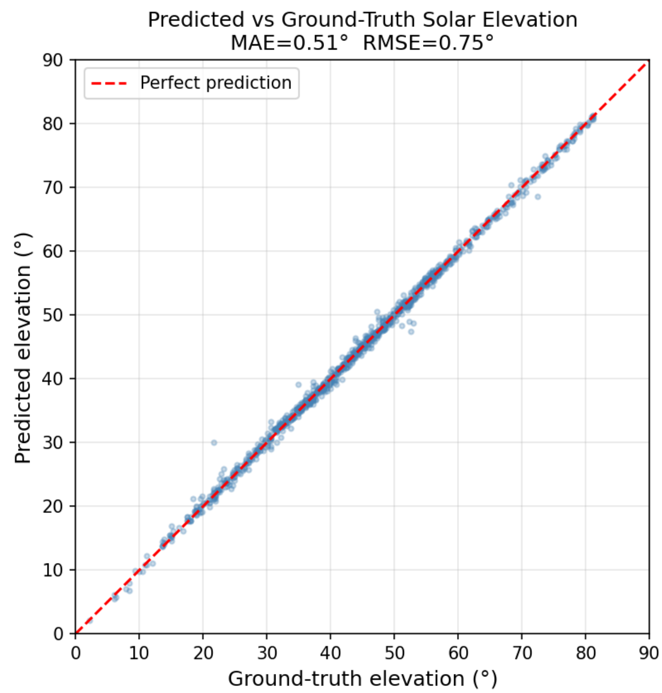

Fig. 3: Ground truth elevation vs. predicted elevation

Fig. 2. We find a minimum MAE error of $0.27°$ for elevations $> 80°$ and a maximum MAE error of $0.58°$ for elevations around $10°$. Fig. 2 demonstrates a downward slope for error as elevation increases, which tells us the model performs better on high solar elevation. This finding is discussed further is section IV-D.

TABLE II: Elevation estimation accuracy (test set)

Metric	Result
MAE	$0.511°$
RMSE	$0.753°$
Mean bias	$0.069°$
Within $\pm 2°$	98.3%
Within $\pm 5°$	99.8%

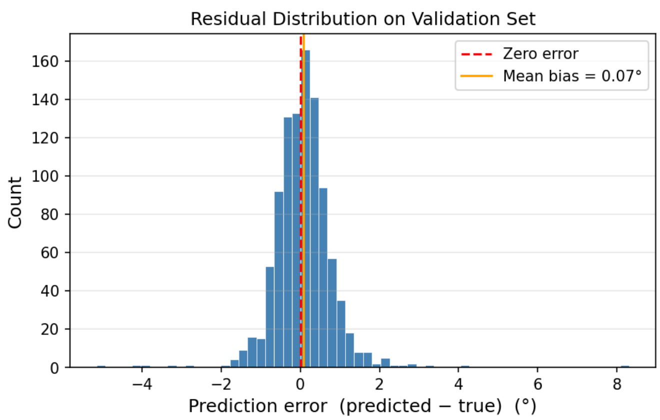

Fig. 4: CNN elevation estimation performance on the test set.

C. Geolocation Accuracy

23 images from Christchurch New Zealand and 25 images from Sydney Australia were sourced online using the tool *Mapillary* [15]. Mapillary provides crowd sourced, street view style images as well as timestamps per image.

The geolocation pipeline was assessed on each set and resulting locations given in Figure. 5.

The result show a high degree of error, with a distance of 2,300Km between the predicted location and CHristchurch.

Each set was split into 3 unique sets with an equal number of images and similar results were found. Curiously, the pipeline showed very low accuracy, but high precision. All estimates were far off the true location, but quite close to eachother.

This result must come from either the CNN predicting similar elevations across the image set or from consistant timestamp inaccuracy of the given set.

Assuming the former, implies that the CNN does not generalise on Mapillary's dataset, which is interesting in and of itself as both Google street view and Mapillary provide the same style of images. Further research would be required to find out why this would occur.

The latter implies that Mapillary's dataset is susceptible to timestamp noise. This may be the case, but does not explain the high precision of error.

This discrepancy remains unknown, but it is assumed to be caused by timestamp inaccuracy. This is covered more in section V.

D. Qualitative Analysis

We analyse an interesting result of the CNN elevation estimation. Fig. 6 shows 2 near identical images. Image (a) includes a hedge on the left edge, while image (b) only includes the hedges shadow. Image (a) has an estimated elevation of $38.6°$, only $4.1°$ off of the true elevation at the time of $34.5°$. Image (b) was estimated at $25.05°$ giving an error of $9.45°$.

We predict that the CNN is able to use the height of an object and the length of its shadow to estimate elevation. When only a shadow exists with no source of the shadow, the network has less information to base its guess on, and hence results in a worse estimation.

(a) Christchurch, New Zealand (b) Sydney, Australia

Fig. 5: Geolocation results for two locations

(a) $38.6°$ CNN Estimation (b) $25.05°$ CNN Estimation

Fig. 6: Near-Identical images with differing elevation results. True elevation = $34.5°$

Referring to Fig. 2, We see as true elevation increases, MAE error drops. We propose that this finding as well as the observed errors in image (a) and (b) are caused by the same image features.

When solar elevation is higher, the ratio between an objects height and its shadow increases. At the limit, where elevation approaches $90°$, the ratio of object height to shadow length approaches infinity. Inversely, when the solar elevation is low, a shadow stretches quite a distance from the object casting the shadow. This means that at low elevations, for an object which is fully visible in the image, its shadow may stretch outside of the images view, meaning that the CNN is not able to find the ratio between object height and shadow length, hence decreasing accuracy of the CNN's prediction.

V. Discussion

When the CNN was tested against the 1,000 image evaluation set, a high degree of generalisation was observed. Fig. 3 shows a near perfect lineal association between ground truth and predicted elevation. As well as a mean bias of $+0.07°$ with 98.3% of images being within $\pm 2°$ of ground truth. However, when tested against images taken from other sources such as Mapillary, this accuracy fails to generalise.

When the geolocation pipeline is tested, a high error is observed with a best case error of 2,300 Km. We propose two sources of this error: Noisy input timestamps, and failure for the CNN to generalise.

Great difficulty was found when sourcing images with metadata containing location and an accurate timestamp. The results in a noisy input for the geolocation pipeline, hence why only a few dataset were used. The effect of this is images with incorrect timestamps or non-ideal conditions.

Correct timestamping is crucial for the geolocation pipeline to function, any inaccuracy results in thousands of kilometers of error. Many cases of incorrect timestamps were observed when curating a test set from the Mapillary repository, some images stating the time was midnight while the image clearly shows a sunny scene.

The effect of this is that no timestamp can be fully trusted from this source. If an image was suposedly taken at 4pm and the image seems to support this, the outliers with incorrect timestamps suggest that the seemingly correct image may be still giving false information.

The other source of error is based on the training data for the CNN model.

The CNN favours images with clear skies and clear shadows, hence taking a random set of images is not feasible. The images must be curated to select for ideal conditions. This is a shortcoming of the training data acquisition pipeline. The dataset used to train the model must be manually labeled, however, a manual label cannot be given for an image with no clear sun. Hence images of overcast days, or images where the sun is occluded were not present in training data leading to generalisation for a domain of perfect conditions.

Future work would try to expand the training set to include all image conditions. If an accurate timestamp and location of an image is given, the elevation of the sun can be directly calculated. However, as mentioned above, this dataset cannot be found on the internet.

VI. Conclusion

This paper shows that it is possible for a Convolutional Neural Network to accurately predict the elevation of the sun in a scene with a high degree of accuracy and generalisation. When conditions are right, the model can accurately predict the elevation of 98.3% images within $\pm 2^{\circ}$. However, using this to accurately geolocate images still remains an open issue. The major problem we faced is that of finding a very general training set. As it stands, the CNN was only trained on perfect conditions and hence accuracy falls when images outside of this domain are provided.

Future work may include using a synthetic dataset for course training, and fine tuning the model using the dataset used in this paper, leading to a more robust model.

References

[1] D. Moreira, A. Bharati, J. Brogan, A. Pinto, M. Parowski, K. W. Bowyer, P. J. Flynn, A. Rocha, and W. J. Scheirer, "Image provenance analysis at scale," *IEEE Transactions on Image Processing*, vol. 27, no. 12, pp. 6109–6123, 2018.

[2] G. Friedland and R. Sommer, "Cybercasing the Joint: On the Privacy Implications of Geo-Tagging."

[3] J. Han and S. Lee, "A study on the processing of timestamps in the creation of multimedia files on mobile devices," *Journal of Information Processing Systems*, vol. 18, no. 3, pp. 402–410, 2022.

[4] J. Hays and A. A. Efros, "Im2gps: estimating geographic information from a single image," in *2008 IEEE Conference on Computer Vision and Pattern Recognition*, 2008, pp. 1–8.

[5] T. Weyand, I. Kostrikov, and J. Philbin, "PlaNet - Photo Geolocation with Convolutional Neural Networks," in *Computer Vision – ECCV 2016*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds. Cham: Springer International Publishing, 2016, vol. 9912, pp. 37–55.

[6] Y. Liu, J. Ding, G. Deng, Y. Li, T. Zhang, W. Sun, Y. Zheng, J. Ge, and Y. Liu, "Image-Based Geolocation Using Large Vision-Language Models."

[7] Z. Wang, D. Xu, R. M. S. Khan, Y. Lin, Z. Fan, and X. Zhu, "Llmgeo: Benchmarking large language models on image geolocation in-the-wild," 2024. [Online]. Available: https://arxiv.org/abs/2405.20363

[8] OpenAI and J. A. et al, "Gpt-4 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2303.08774

[9] F. E. Sandnes, "Determining the Geographical Location of Image Scenes based on Object Shadow Lengths," *Journal of Signal Processing Systems*, vol. 65, no. 1, pp. 35–47, Oct. 2011.

[10] J. Choi, C.-W. Wong, A. Hajj-Ahmad, M. Wu, and Y. Ren, "Invisible Geolocation Signature Extraction From a Single Image," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2598–2613, 2022.

[11] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks."

[12] J. Deng, W. Dong, R. Socher, L.-J. Li, Kai Li, and Li Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*. Miami, FL: IEEE, Jun. 2009, pp. 248–255.

[13] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" Nov. 2014.

[14] J. Djolonga, J. Yung, M. Tschannen, R. Romijnders, L. Beyer, A. Kolesnikov, J. Puigcerver, M. Minderer, A. D'Amour, D. Moldovan, S. Gelly, N. Houlsby, X. Zhai, and M. Lucic, "On robustness and transferability of convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 16 458–16 468.

[15] G. Neuhold, T. Ollmann, S. R. Bulo, and P. Kontschieder, "The Mapillary Vistas Dataset for Semantic Understanding of Street Scenes," in *2017 IEEE International Conference on Computer Vision (ICCV)*. Venice: IEEE, Oct. 2017, pp. 5000–5009.